

Flask Notes

Introduction to Flask

Flask is a lightweight Python web framework used to develop:

- Websites
- Web applications
- REST APIs

Flask is simple, fast, and beginner friendly.

Features of Flask

- Lightweight framework
 - Easy to learn
 - Built-in development server
 - Supports routing
 - Supports templates (HTML)
 - Supports static files (CSS, JS, Images)
 - Easy database integration
-

Recommended Flask Project Structure

A good project structure keeps the application organized and easy to maintain.

```
□ project_name/  
|  
├─ app.py  
|  
├─ templates/  
|   ├─ index.html  
|   └─ about.html
```

```
|   └─ contact.html
|
|   └─ static/
|       └─ css/
|           └─ style.css
|           |
|           └─ js/
|               └─ index.js
|               |
|               └─ images/
|                   └─ image1.jpg
|                   └─ image2.jpg
|
|   └─ venv/
|
└─ requirements.txt
□
```

Explanation of Each File and Folder

1. app.py

Main file of the Flask application.

Used for:

- Creating Flask app
- Defining routes
- Running the application

2. templates/

Stores HTML files.

Examples:

- index.html
- about.html
- contact.html

Flask automatically searches for HTML files inside the *templates* folder.

3. static/

Stores static files such as:

- CSS files
- JavaScript files
- Images

Example:

```
□ static/  
|  
├─ css/  
├─ js/  
└─ images/  
□
```

4. requirements.txt

Used to store project dependencies/packages.

Example:

```
□ Flask  
mysql-connector-python
```

□ Install dependencies using:

```
□ pip install -r requirements.txt  
□
```

Basic Steps to Develop Flask Application

1. Import Flask class
 2. Create Flask application object
 3. Define routes
 4. Run the application
-

Step-by-Step Flask Setup

Step 1: Create Project Folder

Create a project folder and open it in Visual Studio Code.

Example:

❑ *FlaskProject*

❑

Step 2: Open Terminal

Open terminal inside VS Code.

Shortcut:

❑ *Ctrl + `*

❑

Step 3: Create Virtual Environment

Command

❑ *python -m venv venv*

□ Explanation

- *python* → Python interpreter
 - *-m* → Run module
 - *venv* → Virtual environment module
 - second *venv* → Folder name
-

Step 4: Activate Virtual Environment

Windows

□ *venv\Scripts\activate*

□ After activation:

□ *(venv)*

□ appears in terminal.

Step 5: Install Flask

Command

□ *pip install flask*

□

Step 6: Verify Flask Installation

□ *pip list*

□ You can see:

□ *Flask*



What is Virtual Environment?

A virtual environment creates an isolated Python environment for each project.

Advantages

- Avoids dependency conflicts
 - Keeps project packages separate
 - Easy project management
-

Flask Application Creation

Step 1: Import Flask

 `from flask import Flask`

Explanation

- `flask` → Module name
 - `Flask` → Class name
-

Step 2: Create Flask Application Object

 `app = Flask(__name__)`



What is `__name__`?

`__name__` is a special Python variable.

It tells Flask:

- where the current file is located
 - where to find templates folder
 - where to find static folder
-

Understanding Class and Object

Example

```
□ class Student:  
    pass
```

```
s = Student()
```

□ Explanation

Concept	Example
Class	Student
Object	s
Object Creation	Student()

Similarly:

```
□ app = Flask(__name__)  
□
```

Concept Example

Class	Flask
-------	-------

Object	app
--------	-----

What is a Route?

A route connects:

- URL
with
 - Python function
-

Route Syntax

```
@app.route('/')  
def home():  
    return "Welcome"  

```

Route Explanation

Part	Description
<code>@app.route('/')</code>	URL mapping
<code>def home()</code>	Function
<code>return</code>	Sends response to browser

Complete Flask Program

```
from flask import Flask  
  
# Create Flask application object
```



```
app = Flask(__name__)

# Home Route
@app.route('/')
def home():
    return "Welcome to Home Page"

# About Route
@app.route('/about')
def about():
    return "Welcome to About Page"

# Contact Route
@app.route('/contact')
def contact():
    return "Welcome to Contact Page"

# Run the Flask application
app.run(debug=True)
```

Explanation of *debug=True*

□ `app.run(debug=True)`

□ Advantages

- Automatically reloads server after code changes
 - Shows error messages clearly
-

Running Flask Application

Command

❑ `python app.py`

❑ Output:

❑ `Running on http://127.0.0.1:5000/`

❑

Access URLs

Route	URL
Home	<code>http://127.0.0.1:5000/</code>
About	<code>http://127.0.0.1:5000/about</code>
Contact	<code>http://127.0.0.1:5000/contact</code>

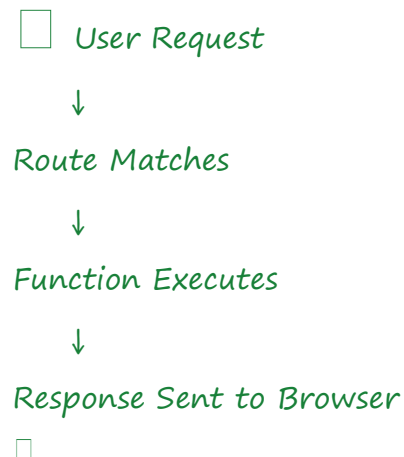
Common Mistakes and Corrections

Wrong	Correct
<code>from flask import flask</code>	<code>from flask import Flask</code>
<code>app=flask(__name__)</code>	<code>app = Flask(__name__)</code>
<code>app.route('/')</code>	<code>@app.route('/')</code>
<code>app-run()</code>	<code>app.run()</code>

```
python -m venv env    python -m venv venv
```

```
pip install flask    pip install flask
```

Flask Request Flow



Important Interview Questions

1. What is Flask?

Flask is a lightweight Python web framework used to build web applications and APIs.

2. Why use Flask?

- Easy to learn
 - Flexible
 - Lightweight
 - Fast development
-

3. What is Routing?

Routing maps URLs to Python functions.

4. What is Virtual Environment?

An isolated environment used to manage project dependencies separately.

5. What is the use of templates folder?

Stores HTML files.

6. What is the use of static folder?

Stores:

- CSS
 - JavaScript
 - Images
-

Final Minimal Flask Program

```
☐ from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')
def home():
```

```
    return "Hello Flask"
```

```
app.run(debug=True)
```

```
☐
```